Top Secret - Homework 6 UML Class Diagram

Relationship Key

- Uses (dashed arrow)
- Implements (dashed arrow, hollow triangle)

TopSecret

+ main(args: String[]): void

TopSecret.main() creates the model objects and the Controller, then starts TerminalView (default) or WebServer on port 3002 when run with "-web".

TerminalView

- controller: ControllerInterface
- scnr: Scanner
- currentUser: String

+ TerminalView(controller: ControllerInterface)
- display(message: String): void
- getInput(message: String): String
- getRequiredInput(message: String): String
- getOptionalInput(message: String): String
- getInputInt(min: int, max: int, message: String): int
- getInputInt(message: String, min: int, max: int, othernums: int...): int
- getPositiveInt(message: String): int
- isAdmin(): boolean
- capitalize(value: String): String
+ run(): int
- loginStep(): boolean
- promptSignup(): String[]
- promptChangePassword(): String[]
- changePassword(): void
- showMainMenu(): void
- browseMenu(itemType: String): void
- listEntries(itemType: String): void
- readEntry(itemType: String): void
- addEntry(itemType: String): void
- editEntry(itemType: String): void
- promptRecordFields(itemType String): List<String>
- addRequiredField(fields: List<String>, message: String): boolean
- assignAgentToMission(): void
- createUser(): void
- manageAccess(): void
- reviewAuditLog(): void
- browseMessages(): void
- sendMessage(): void

WebServer

- controller: ControllerInterface
- sessions: Map<String, String>
- port: int
- server: HttpServer
- view: WebView

+ WebServer(controller: ControllerInterface, port: int)
+ getPort(): int
+ start(): void
+ stop(): void

WebView

+ WebView()
+ generateHTML(title: String, text: String): String
+ mainMenuView(controllerMenu: String): String
+ browseView(type: String, records: String): String
+ readView(type: String, record: String): String
+ auditView(logs: String): String
- getTypeFromTitle(title: String): String

HTMLUtils

- HTMLUtils()
+ escapeHTML(text: String): String
+ createPage(title: String, body: String): String
+ createHeading(heading: String): String
+ createTextBlock(text: String): String
+ createLink(url: String, label: String): String
+ createLinkList(links: String[][]): String
+ createIDForm(action: String, label: String): String
+ createReadForm(type: String): String
+ createBackLink(): String

Controller

- users: UserManagerInterface
- access: AccessControlInterface
- store: DataStoreInterface
- audit: AuditLogInterface
- messages: MessageManager

+ Controller(users: UserManagerInterface, access: AccessControlInterface, store: DataStoreInterface, audit: AuditLogInterface, messages: MessageManager)
+ hasUsers(): boolean
+ verifyLogin(username: String, password: String): boolean
+ needsPasswordSetup(username: String): boolean
+ createFirstAccount(username: String, password1: String, passwordConfirm: String): String
+ changePassword(username: String, password1: String, passwordConfirm: String): String
+ createUser(adminUsername: String, newUsername: String): String
+ manageAccess(adminUsername: String, targetUsername: String, action: String): String
+ isAdmin(username: String): boolean
+ showMainMenu(username: String): String
+ browseMainMenu(username: String, itemType: String): String
+ listEntries(username: String, itemType: String): String
+ readEntry(username: String, itemType: String, id: int): String
+ addEntry(username: String, itemType: String, fields: List<String>): String
+ editEntry(username: String, itemType: String, id: int, fields: List<String>): String
+ reviewAuditLog(username: String): String
+ assignAgentToMission(username: String, agentId: int, missionId: int): String
+ sendMessage(username: String, agentName: String, message: String): String
+ viewMessages(username: String): String

MessageManager

userAccess: UserManagerInterface
dataStore: DataStoreInterface

+ MessageManager(userAccess: UserManagerInterface, dataStore: DataStoreInterface)
+ createMessage(recipient: Agent, contents: String): Message
+ retrieveMessages(agent: Agent): List<Message>

Cipher

- substitutions: HashMap<Character, Character>

+ Cipher(keyFileName: String)
+ decipher(cipherText: String): String
+ encipher(plaintext: String): String

FileHandler

+ getFileLinesList(fileName: String, fileType: FileTypes): ArrayList<String>
+ listDataFiles(): ArrayList<String>

«enumeration»
FileTypes

DATA
CIPHER

UserManager

- CREDENTIAL_FILE: Path
- KEY_FILE: String
- ALLOWED_CHARACTERS: String
- dataStore: DataStoreInterface

+ UserManager(dataStore: DataStoreInterface)
- getNextUserID(): int
~ isUsernameValid(username: String): boolean
~ isPasswordValid(password: String): boolean
- loadCredentials(): Map<String, String>
- saveCredentials(credentials: Map<String, String>): void
+ hasUser(): boolean
+ createUser(username: String): User
+ getUserByID(userID: int): User
+ verifyLogin(username: String, password: String): boolean
+ changePassword(username: String, newPassword1: String, newPasswordConfirmation: String): void
+ needsPasswordSetup(username: String): boolean
+ getUserByUsername(username: String): User
+ listUsernames(): List<String>

AccessControl

- admins: Set<String>
- users: Set<String>
- rolesFile: Path

+ AccessControl(admins: Set<String>, users: Set<String>)
+ AccessControl(admins: Set<String>, users: Set<String>, rolesFile: Path)
- loadRoles(file: Path, adminsOut: Set<String>, usersOut: Set<String>): boolean
- saveRoles(): void
+ isAdmin(username: String): boolean
+ promoteUser(adminUsername: String, targetUsername: String): boolean
+ demoteUser(adminUsername: String, targetUsername: String): boolean
+ hasPermission(username: String, action: String): boolean

AuditLog

- dataStore: DataStoreInterface

+ AuditLog(dataStore: DataStoreInterface)
+ log(action: LoggedAction): LoggedAction
+ getLog(): List<LoggedAction>

SQLiteDataStore

- DATABASE_URL: String
- databaseUrl: String
- connection: Connection

+ SQLiteDataStore()
+ SQLiteDataStore(databaseUrl: String)
+ initDatabase(): void
+ importDatasetTsv(fileName: String, table: String): void
- importTsvReader(reader: BufferedReader, table: String): void
+ close(): void
+ addMission(mission: Mission): Mission
+ updateMission(mission: Mission): Mission
+ listAgents(): List<Agent>
+ listFacilities(): List<Facility>
+ listBriefs(): List<Brief>
+ getAgent(id: int): Agent
+ getFacility(id: int): Facility
+ getBrief(id: int): Brief
+ addAgent(agent: Agent): Agent
+ updateAgent(agent: Agent): Agent
+ addFacility(facility: Facility): Facility
+ updateFacility(facility: Facility): Facility
+ addBrief(brief: Brief): Brief
+ updateBrief(brief: Brief): Brief
+ listLogs(): List<LoggedAction>
+ listLogs(userID: int): List<LoggedAction>
+ addLog(log: LoggedAction): LoggedAction
+ listUsers(): List<User>
+ getUser(userID: int): User
+ addUser(user: User): User
+ listMissions(): List<Mission>
+ getMission(id: int): Mission
+ assignAgentToMission(agentID: int, missionID: int): void
+ unassignAgentFromMission(agentID: int, missionID: int): void
+ listMissionsForAgent(agentID: int): List<Mission>
+ listAgentsForMission(missionID: int): List<Agent>
+ listBriefsForMission(missionID: int): List<Brief>
+ addMessage(newMessage: Message): Message
+ listMessagesForAgent(recipient: Agent): List<Message>

«abstract»
View

~ *singleMissionView(mission: Mission): String*
~ *listMissionsView(missions: List<Mission>): String*
~ *singleFacilityView(facility: Facility): String*
~ *listFacilitiesView(facilities: List<Facility>): String*
~ *singleBriefView(brief: Brief): String*
~ *listBriefsView(briefs: List<Brief>): String*
~ *singleAgentView(agent: Agent): String*
~ *listAgentsView(agents: List<Agent>): String*
~ *singleUserView(user: User): String*
~ *listUsersView(users: List<User>): String*
~ *singleLogView(log: LoggedAction): String*
~ *listLogsView(logs: List<LoggedAction>): String*
~ *mainMenuView(): String*
~ *browseMissionsView(): String*
~ *browseFacilitiesView(): String*
~ *browseBriefsView(): String*
~ *browseAgentsView(): String*

«interface»
UserInterfaceInterface

+ run(): int
+ loginStep(): boolean
+ showMainMenu(): void
+ browseMenu(itemType: String): void
+ listEntries(itemType: String): void
+ readEntry(itemType: String): void
+ addEntry(itemType: String): void
+ reviewAuditLog(): void
+ changePassword(): void
+ createUser(): void
+ manageAccess(): void

«interface»
ControllerInterface

+ hasUsers(): boolean
+ verifyLogin(username: String, password: String): boolean
+ needsPasswordSetup(username: String): boolean
+ createFirstAccount(username: String, password1: String, passwordConfirm: String): String
+ changePassword(username: String, password1: String, passwordConfirm: String): String
+ createUser(adminUsername: String, newUsername: String): String
+ manageAccess(adminUsername: String, targetUsername: String, action: String): String
+ listUsernames(username: String): String
+ isAdmin(username: String): boolean
+ showMainMenu(username: String): String
+ browseMainMenu(username: String, itemType: String): String
+ listEntries(username: String, itemType: String): String
+ readEntry(username: String, itemType: String, id: int): String
+ addEntry(username: String, itemType: String, fields: List<String>): String
+ editEntry(username: String, itemType: String, id: int, fields: List<String>): String
+ reviewAuditLog(username: String): String
+ assignAgentToMission(username: String, agentId: int, missionId: int): String
+ sendMessage(username: String, agentName: String, message: String): String
+ viewMessages(username: String): String

UserInterface

- users: UserManagerInterface
- access: AccessControlInterface
- store: DataStoreInterface
- audit: AuditLogInterface
- sc: Scanner
- currentUser: String
- currentUserObj: User

+ UserInterface(users: UserManagerInterface, access: AccessControlInterface, store: DataStoreInterface, audit: AuditLogInterface)
+ run(): int
+ loginStep(): boolean
+ showMainMenu(): void
+ browseMenu(itemType: String): void
+ listEntries(itemType: String): void
+ readEntry(itemType: String): void
+ addEntry(itemType: String): void
+ reviewAuditLog(): void
+ changePassword(): void
+ createUser(): void
+ manageAccess(): void

Legacy HW5 terminal UI. Still in the code, but TopSecret now uses TerminalView + Controller instead.

«interface»
UserManagerInterface

+ hasUser(): boolean
+ createUser(username: String): User
+ getUserByID(userID: int): User
+ verifyLogin(username: String, Password: String): boolean
+ changePassword(username: String, newPassword1: String, newPasswordConfirmation: String): void
+ needsPasswordSetup(username: String): boolean
+ getUserByUsername(username: String): User
+ listUsernames(): List<String>

«interface»
AccessControlInterface

+ isAdmin(username: String): boolean
+ promoteUser(adminUsername: String, targetUsername: String): boolean
+ demoteUser(adminUsername: String, targetUsername: String): boolean
+ hasPermission(username: String, action: String): boolean

«interface»
AuditLogInterface

+ log(action: LoggedAction): LoggedAction
+ getLog(): List<LoggedAction>

«interface»
DataStoreInterface

+ listMissions(): List<Mission>
+ getMission(id: int): Mission
+ assignAgentToMission(agentID: int, missionID: int): void
+ unassignAgentFromMission(agentID: int, missionID: int): void
+ listMissionsForAgent(agentID: int): List<Mission>
+ listAgentsForMission(missionID: int): List<Agent>
+ listBriefsForMission(missionID: int): List<Brief>
+ initDatabase(): void
+ importDatasetTsv(fileName: String, table: String): void
+ addMission(mission: Mission): Mission
+ updateMission(mission: Mission): Mission
+ listAgents(): List<Agent>
+ listFacilities(): List<Facility>
+ listBriefs(): List<Brief>
+ getAgent(id: int): Agent
+ getFacility(id: int): Facility
+ getBrief(id: int): Brief
+ addAgent(agent: Agent): Agent
+ updateAgent(agent: Agent): Agent
+ addFacility(facility: Facility): Facility
+ updateFacility(facility: Facility): Facility
+ addBrief(brief: Brief): Brief
+ updateBrief(brief: Brief): Brief
+ listLogs(): List<LoggedAction>
+ listLogs(userID: int): List<LoggedAction>
+ addLog(log: LoggedAction): LoggedAction
+ listUsers(): List<User>
+ getUser(userID: int): User
+ addUser(user: User): User
+ addMessage(newMessage: Message): Message
+ listMessagesForAgent(recipient: Agent): List<Message>

TopSecretException

+ TopSecretException(message: String)
+ TopSecretException(message: String, cause: Throwable)

TopSecretException is thrown by Cipher, UserManager and MessageManager, and caught in Controller and UserInterface.

Standard users:
- Read briefs, agents, facilities, missions
- View and send messages

Admins can also:
- Add and edit entries
- Create users, assign agents
- Review the audit log

First account is an admin. Admins can promote standard users and demote other admins, but cannot demote themselves.

utility

+ missionToStr(m: Mission): String
+ facilityToStr(f: Facility): String
+ briefToStr(b: Brief): String
+ agentToStr(a: Agent): String
+ userToStr(user: User): String
+ logToStr(log: LoggedAction): String
+ messageToStr(m: Message): String
+ missionsToStr(missions: List<Mission>): List<String>
+ facilitiesToStr(facilities: List<Facility>): List<String>
+ briefsToStr(briefs: List<Brief>): List<String>
+ agentsToStr(agents: List<Agent>): List<String>
+ usersToStr(users: List<User>): List<String>
+ logsToStr(logs: List<LoggedAction>): List<String>
+ messagesToStr(messages: List<Message>): List<String>

View (abstract) and utility are in the code but are not used by other classes yet.

Data classes (model package)

Brief

- briefID: int
- missionID: int
- title: String
- date: String
- content: String

+ Brief(missionID: int, title: String, date: String, content: String)
+ Brief(briefID: int, missionID: int, title: String, date: String, content: String)
+ getContent(): String
+ getDate(): String
- setDate(date: String): void
+ getTitle(): String
+ getMissionID(): int
+ getBriefID(): int

Mission

- missionID: int
- facilityID: int
- title: String

+ Mission(title: String, facilityID: int)
+ Mission(missionID: int, title: String, facilityID: int)
+ getMissionID(): int
+ getTitle(): String
+ getFacilityID(): int

Facility

- facilityID: int
- name: String
- abbreviation: String

+ Facility(name: String, abbreviation: String)
+ Facility(facilityID: int, name: String, abbreviation: String)
+ getName(): String
+ getAbbreviation(): String
+ getFacilityID(): int

Agent

- agentID: int
- facilityID: int
- missionID: Integer
- name: String
- date_of_birth: String
- date_of_death: String
- notes: String

+ Agent(missionID: Integer, facilityID: int, name: String, date_of_birth: String, date_of_death: String, notes: String)
+ Agent(agentID: int, missionID: Integer, facilityID: int, name: String, date_of_birth: String, date_of_death: String, notes: String)
+ getFacilityID(): int
+ getMissionID(): Integer
+ getAgentID(): int
+ getNotes(): String
+ getDate_of_death(): String
+ setDate_of_death(date_of_death: String): void
+ getDate_of_birth(): String
+ setDate_of_birth(date_of_birth: String): void
+ getName(): String

Message

- recipient: Agent
- content: String

+ Message(recipient: Agent, content: String)
+ getMessageContent(): String
+ getRecipient(): Agent

User

- userID: int
- username: String

+ User(userID: int, username: String)
+ getUserID(): int
+ getUsername(): String

LoggedAction

+ READ: int
+ CREATE: int
- action: int
- user: User
- dateTime: String
- record: String

+ LoggedAction(action: int, user: User, dateTime: String, record: String)
+ getAction(): int
+ getUser(): User
+ getDateTime(): String
+ getRecord(): String

Data classes are returned by DataStoreInterface / SQLiteDataStore and passed through TopSecret, Controller, MessageManager, UserInterface, View and utility.